



Distributed Systems

Time in Distributed Systems



Recap

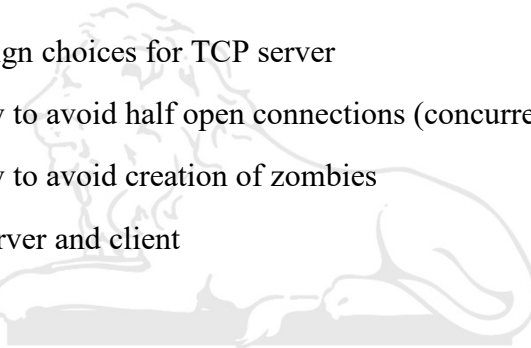


TCP echo server and client

- Design choices for TCP server
- How to avoid half open connections (concurrent server)
- How to avoid creation of zombies

UDP echo server and client

Unix signals



Clock Synchronization



- Each process P_i has a **state** s_i
- The state of the process includes the values of all its variables and objects in the system that it affects
- As a process executes it takes a series of actions (events) that change its state
- The sequence of events within a single process (and multiple processes on a **single machine**) can be totally ordered
- **Multiple machines** with physical clocks in a Distributed System. How can we keep them more or less synchronized?

IIT ROORKEE ■ ■ ■

How clocks work

- Computer clocks (timer) ^{RTC} use crystals that oscillate at a certain frequency
- Every H oscillations, the timer chip interrupts once (clock tick).
- No. of interrupts per second is typically 18.2, 50, 60, 100
- The interrupt handler increments a counter that keeps track of no. of ticks from a reference in the past (~~epoch~~) Unix 1 Jan 1970
- Knowing no. of ticks per second, we can calculate year, month, day, time of day etc.

Clock Drift

- Unfortunately, period of crystal oscillation varies slightly
- If it oscillates faster, more ticks per real second, so clock runs faster; similar for slower clocks
- For machine p , when correct time is t , let machine clock show time as $C = C_p(t)$
- Ideally, $C_p(t) = t$ for all p, t
- In practice,

$$1 - \rho \leq dC/dt \leq 1 + \rho$$
- ρ = max. clock drift rate, usually around 10^{-5} for cheap oscillators
- Skew between clocks (difference in clock values of two machines)

- **Clock drift rate** – Change in the offset between the clock and reference clock per unit time
- **Skew** – Instantaneous difference between the clock

Resynchronization

- Periodic resynchronization needed to offset skew
- If two clocks are drifting in opposite directions, max. skew after time t is $2 \rho t$
- If application requires that clock skew $< \delta$, then resynchronization period (r) $2\rho r \leq \delta \implies r \leq \frac{\delta}{2\rho}$
- Usually ρ and δ are known

- GMT: Greenwich Mean Time
- Astronomical time
- TAI: International Atomic Time
- UTC: Universal Coordinated Time $\text{--- TAI} + \text{leap sec.}$
 - Primary time standard by which the world regulates clocks and time
 - Maintained by the Bureau International des Poids et Mesures (BIPM) - International Bureau of Weights and Measures
- IST: Indian standard time

Coordinated Universal Time (UTC) time is broadcasted from different sources around the world, ex.

- National Institute of Standards & Technology (NIST) – runs radio stations, most famous being WWV, anyone with a proper receiver can tune in
- United States Naval Observatory (USNO) – supplies time to all defense sources, among others
- National Physical Laboratory in UK
- GPS satellites
- Many others

Clock Synchronization (synchronous system)



- The time to execute each step of a process has known lower and upper bounds
- Each message transmitted over a channel is received within a known bounded time (max, min)
- Uncertainty in message transmission time (u) = $\text{max} - \text{min}$
- Each process has a local clock whose drift rate from real time has a known bound

$$t, t' \quad t' = t + \frac{\text{max} + \text{min}}{2}$$

Clock Synchronization



- Internal vs. External synchronization
- Centralized vs. Distributed
- Perfect synchronization not possible because of variable communication delays
- Even synchronization within a bound can not be guaranteed with certainty because of unpredictability of communication delays.
- But still useful !! Ex. – Kerberos, GPS

IIT ROORKEE

11

Cristian's Algorithm (1989)



- One m/c acts as the time server (that receives signals from a source of UTC)
- Each m/c sends a message periodically (**within resynchronization period r**) asking for current time
- Time server replies with its time T
- Sender sets its clock to the reply to $T + \text{RTT}/2$
- Accuracy (if value of min time is known)

$$m_2 \leftarrow \min$$

$$m_1 \leftarrow \min$$

$$T + \min$$

$$T + \text{RTT} - \min$$

$$\pm \left(\frac{\text{RTT}}{2} - \min \right)$$



IIT ROORKEE

12

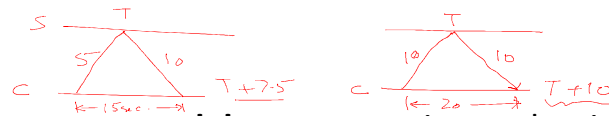
Cristian's Algorithm



- The method achieves synchronization only if the observed RTT between client and server are sufficiently short compared with the required accuracy
- Problems:
 - Processing time at server
 - use knowledge of processing time at server if known
 - message delay
 - server time is less than sender's current time

IIT ROORKEE

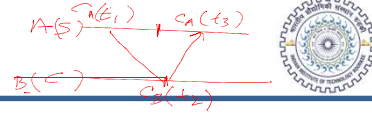
13



- **Handling message delay:** try to estimate the time the message with the timer server's time took to reach the sender
 - ✓ 1. measure round trip time and halve it
 2. make multiple measurements and take minimum
 3. make multiple measurements of round trip time, discard too high (or low) values, take average of rest
- **Handling fast clocks**

do not set clock backwards; slow it down over a period of time to bring in tune with server's clock

Berkeley Algorithm (TEMPO)



- Centralized as in Cristian's, but the time server is active (no WWV receiver)
 1. Time server (A) tells its own time and asks for time of other m/cs (B) at periodic intervals
 2. Time server estimates the difference between the clocks of the machines A and B by observing the RTT of the messages
 3. Estimate difference between the clocks of A and B

$$\frac{C_A(t_1) + C_A(t_3)}{2} - C_B(t_2) \leftarrow \text{Time of m/c B at time } t_2$$

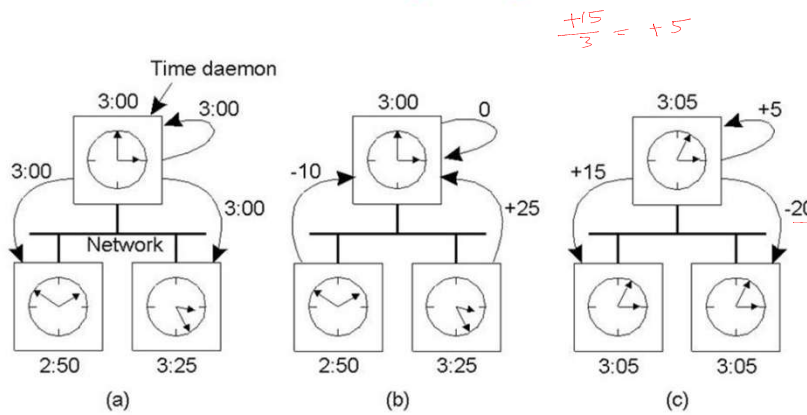
IIT ROORKEE

15

Berkeley Algorithm

3. time server averages the times (?) and sends the value by which clock should be adjusted to slaves
 4. M/cs set their time (advances immediately or slows down slowly) to the new time
- Estimation of transmission delay (RTT) as before
 - Uses an election algorithm that elects a new master should the machine running the current master crash

The Berkeley Algorithm



Distributed Systems by A S Tanenbaum and M V Steen

Berkeley Algorithm

■ Accuracy ϵ

- Let T_{mAB} and T_{mBA} be minimal possible transmission times from **A** to **B** and from **B** to **A**

- Let $T_M \geq 2 * \max(T_{mAB}, T_{mBA})$ s.t. $C_A(t_3) - C_A(t_1) \leq T_M$

(T_M is the bound on RTT)

$$\epsilon = \frac{T_M - 2 \min(T_{mAB}, T_{mBA})}{2} \geq 0.$$

$\frac{RTT}{2} = \min$

ϵ = maximum error in the estimation of time by daemon

“The Accuracy of the Clock Synchronization Achieved by TEMPO in Berkeley UNIX 4.3BSD” by R Gusella and S Zatti in IEEE TRANSACTIONS ON SOFTWARE ENGINEERING. VOL. 15, NO. 7, JULY 1989

External Synchronization

- Clocks must be synchronized with ^{VTC}real time
- Cristian's algorithm can be used if the time server is synchronized with real time
- Berkeley algorithm cannot be used
- Protocol for time synchronization in the Internet?

NTP : Network Time Protocol

SNTP

by David L. Mills

- NTPv1 published in 1988 (RFC 1059)
- NTPv2 published in 1989 (RFC 1119)
- NTPv3 published in 1992 (RFC 1305)
- NTPv4 published in 2010 (RFC 5905)
- **Network Time Security** (NTS) is a secure version of NTPv4 (2020)